CVS Health

Transformer Embeddings

Pritha Ghosh

Daniel Xing

Matt Anderson

January 2026

Executive Summary

Current Reality

- Transformer Embeddings (TE) have consistently shown **strong performance and efficiency gains** over traditional feature engineering
- However, today's TE ecosystem is **operationally fragmented**, with separate pipelines, refresh cadences, and ownership across LOBs
- The current TE foundation was trained under **legacy constraints** and does not support scalable, governed, cross-LOB adoption

What We've Done

- Established a **unified "latest" TE access layer** across Commercial, Medicaid, and Medicare to unblock evaluation
- Created a **stable consolidated dataset for downstream users** while underlying pipelines remain heterogenous
- Laid the groundwork for **retraining on GCP**, enabling evaluation, migration planning, and future standardization

What's Next

- **Retrain TE model** on a governed, cross-LOB GCP foundation
- Align on **inference strategy, refresh cadence, and migration approach** for downstream models
- Partner with downstream teams to **validate performance, manage adoption, and sunset legacy embeddings**

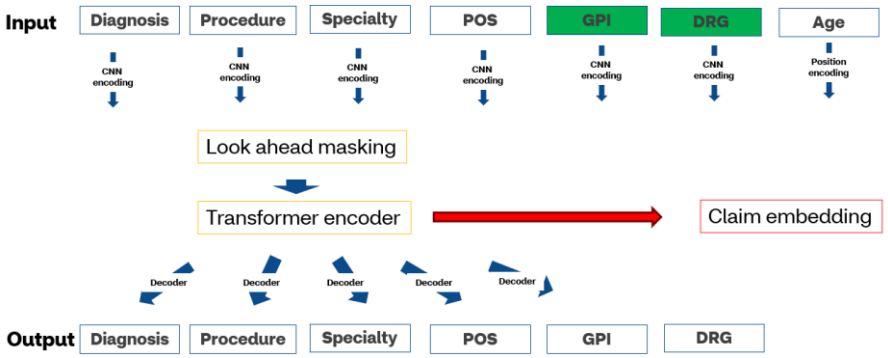
What Are Transformer Embeddings (and why we use them)

Introduction

- Transformer Embeddings (TE) **summarize a member's entire medical history** into a single, dense representation that models sequence, timing, and context – not just counts
- Medical events are treated as a **time-ordered sequence**, not independent rows
- The model learns **relationships across diagnoses, procedures, settings, and time**
- More recent and more relevant events naturally matter more

Why this is better than traditional features

- Captures **order and timing**, not just freq
- Handles **irregular, sparse medical data** naturally
- Reduces hundreds of hand-crafted featu into a **single reusable signal**
- Improves performance across **many downstream models**, not just one

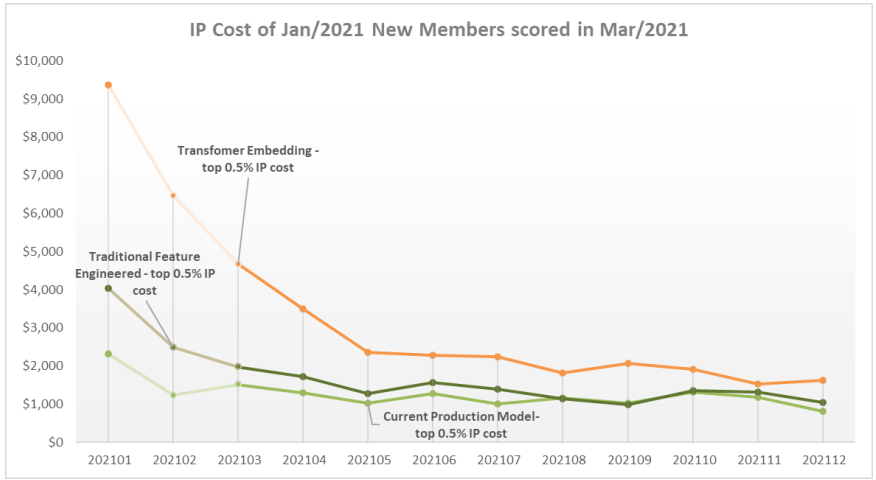


Current Evidence Snapshot

What we've seen so far across prior runs:

- TE models consistently **outperform traditional feature engineering** for high-risk member identification
- TE model identifies members **with higher IP likelihood** and **higher IP/ER** and **overall medical cost**
- Performance gains are **largest at higher-risk deciles**, where business impact is greatest
- **Standardizes feature creation from claims history,** enabling faster iteration and reducing time-to-model for new use cases

Lift at Top 1%	Commercial	Medicaid	Medicare
Traditional FE	13.6	10.2	6.1
TE only	**16.4**	**16.9**	**7.3**



Results from Commercial IP model

Slide courtesy of Jane Zou, Elle Palmer, Zenon Cheng

Current State: Embeddings are Fragmented across LOBs

Commercial

- Nested / array-based embedding storage
- Daily incremental + weekly full refresh
- Stored as a standalone embedding table
- Coupled with RAP model pipeline

Medicaid

- Flat, column-based embedding schema
- Monthly full refresh
- Standalone embedding schema
- Pipeline stable and refreshing as expected

Medicare

- Flat, column-based embedding schema
- Refreshed weekly with IP model scoring
- Embedding subset stored, coupled with other IP model features
- Pipeline execution constrained by compute availability and design (last successful refresh: November 2025)

Key Takeaway: Embeddings exist as LOB-specific artifacts with inconsistent schemas, refresh cadences, and operational reliability – making downstream evaluation and migration difficult

What We Did: Unified “Latest” Embedding Access Layer

Unified Access Layer

- Single consolidated Big Query view in CSDI dataset
- Brings together embeddings from Commercial, Medicaid, and Medicare
- Acts as a single access point for downstream users
- Pipelines remain owned and operated independently by LOB/model teams
- Access layer provides a stable dataset across heterogenous ownership

Design Choice

- Stores only the latest embedding per member
- Fully retrained TE product will be published to the same dataset
- Minimizes downstream change when transitioning to the unified product

Why “Latest” Only (For Now)

- Serves as a bridge, not the final TE product
- Not intended for model training or long-term storage – inference and evaluation only
- Reduces operational fragility while pipelines are heterogeneous
- Avoids unnecessary storage and compute costs during transition
- Enables fast evaluation on current state

Key Takeaway: This layer decouples downstream evaluation from pipeline ownership, simplifying access without changing underlying models, unblocking evaluation and migration planning

Access view here: *edp-prod-storage.edp_ent_transformer_srcv.embedding_combined*

Why Retraining Is Necessary: Rebuilding the Embedding Foundation

Core Reasons for Retraining

Retaining is not a routine model refresh – it is required to rebuild transformer embeddings on a governed, scalable GCP foundation that supports long-term enterprise use

- **Establish a governed and reusable foundation**
  - Current embeddings were trained on **legacy, undocumented pipelines**
  - Training data lineage was not preserved during migration to GCP
- **Improve coverage and population representativeness**
  - Inclusion of **Medicaid** for full population representation
  - Better generalization across **diverse member cohorts**
  - Alignment with **current utilization patterns**, not legacy behavior

Dimension	Current State (V2)	Retrained State (V3)
Data Coverage	Commercial, Medicare from 2016	All LOBs – Commercial, Medicare, Medicaid from 2023
Infrastructure	Legacy Hadoop, undocumented	Fully migrated and governed on GCP
Coverage	Limited data sources	Expanded to include GPI, DRG and National Provider Taxonomy
Governance	None	Full lineage, reproducibility, and metadata tracking

Key Takeaway: To move from a shared access layer to a true shared product, retraining is required to ensure governance, representativeness, and long-term scalability

Training Enablement, GPU Readiness & Cost Guardrails

Why this matters

- Transformer retraining at enterprise scale requires H100-class GPUs
- Not feasible on Vertex AI default GPUs (T4) given model size + runtime constraints
- T4 incompatibility with standard and mainstream model architectures and algorithms

What had to be aligned

- **Business Justification** (why H100 vs alternatives)
- **Training scope & time bounds** (not open-ended compute)
- **Quota + region availability** on governed GCP projects
- **Cost visibility and approvals** before execution

Machine Type	Configuration	Cost	Est. Runtime	Est. Cost	Notes
a3-highgpu-4g	**4 X H100**	**$44/hr**	**~24-36 hrs**	**$1k-$1.5k**	**Bounded run, preferred**
n1-standard-32	4 x T4	$3/hr	~900-1200 hrs	$2.7k - $3.6k	Memory/time constraints

Link to Google Cloud Pricing Calculator

Target Inference & Serving Pipeline

Inference Generation

- Retrained embedding model is used for **batch inference**
- Runs on a **defined cadence or event trigger** (claims-based)
- Embeddings are generated **as of a specific point in time**

Embedding Storage

- Inference outputs are written to **CSDI** dataset
- Stored as **time-anchored snapshots** (as of index date)
- Enables:
  - Evaluation over a fixed cohort
  - Reproducibility of results
  - Auditability

Downstream Consumption

- Downstream models access embeddings via a **stable CSDI interface**
- Consumers are decoupled from pipeline ownership

Operational Guardrails

- Inference jobs are **auditable and executable**
- Backfills and re-runs are **supported without retraining**

Key Takeaway: This pipeline supports a production TE product with phased rollout, ensuring stability and controlled adoption

Embedding Refresh Cadence – Operating Model Decision

Option 1 : Fixed Time-Based Refresh

Weekly or monthly refresh for all members

- Pros
  - Easy to schedule and monitor
- Tradeoffs
  - Recomputes embeddings even when nothing has changed
  - Higher ongoing compute cost

Option 2: Event-Driven Refresh (Recommended Direction)

- Weekly refresh for members with new claims
- Monthly full refresh to capture time-based effects (e.g., aging, recency decay)
- Pros
  - Aligns embeddings with meaningful member state changes
  - Captures temporal dynamics even without new claims
  - Avoids unnecessary recomputation for static members
  - Scales better as population grows
- Tradeoffs
  - More complex orchestration

This is a product operating design that impacts cost, scalability, and downstream stability – and should be aligned across teams

Model Architecture & Training Ablations

Model Architecture and Refactoring

Preserve hierarchical structure

- **Hierarchical structure** for encoding within- (80 codes) and inter-day (200 days)
- Trained to predict **next-day medical codes** (multi-label, 6,297 target groups)

Training & inference efficiency

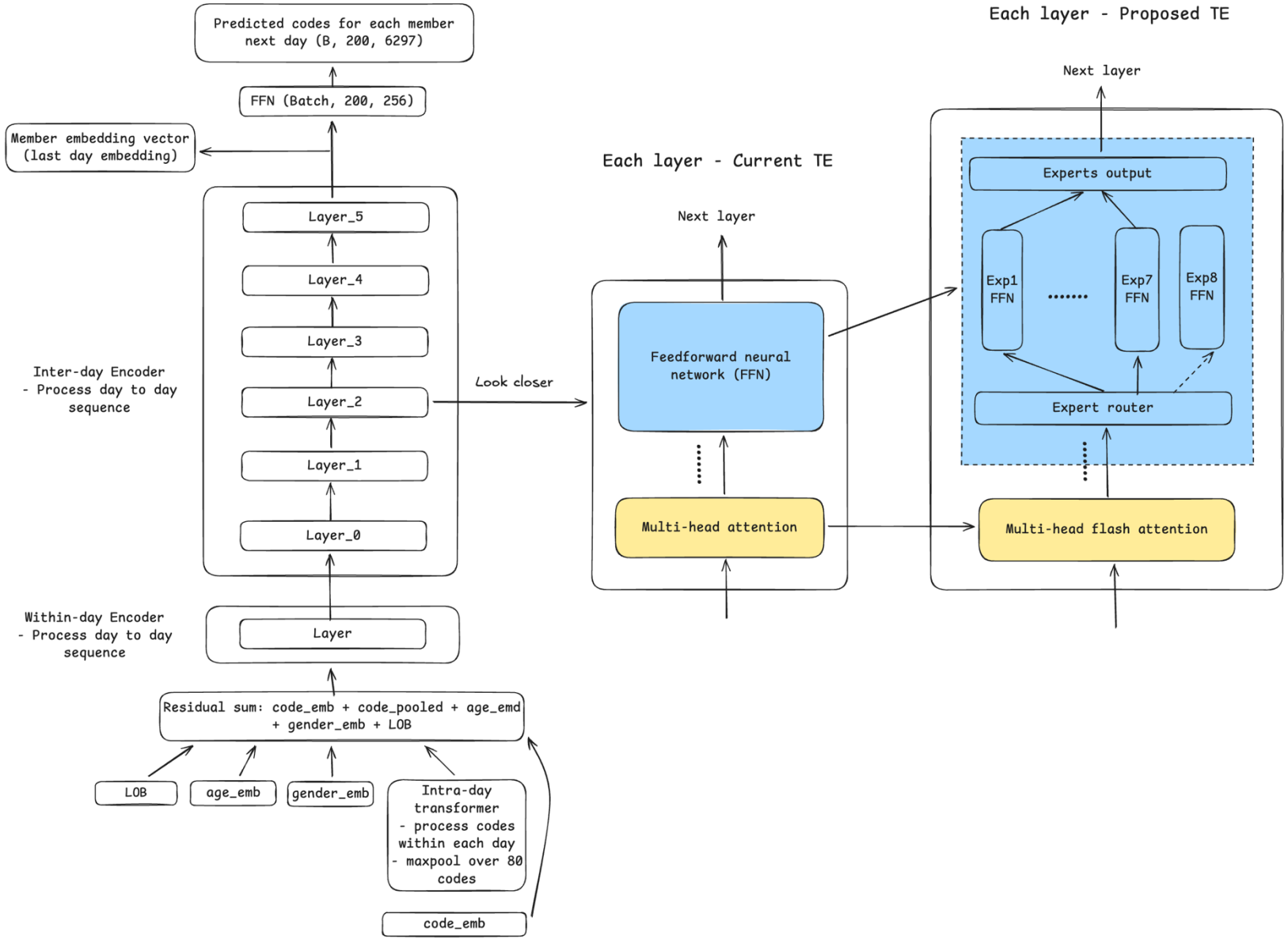
- Replace the attention layer with **flash attention** to avoid GPU IO bottleneck and provides faster training with less GPU memory

Optimize training strategy

- Weighted BCE loss accounting for rare codes
- Enable mixed precision training
- SGD -> Adam learning rate optimizer
- Head dimension 16 -> 32 (fit flash attention)

Specialized embedding representation

- Replace the dense FFN with **mixture of expert**, each token routed to 2 “experts”, improving heterogeneous feature representation without increasing cost



Preliminary Ablation Results (Intrinsic evaluation)

Data

- Commercial: 1.06M
- Medicare: 0.56M
- Medicaid: 0.13M
- Day count > 10
- Time period: 1/1/2023 – 12/31/2023
- Claim lookback period: 36 months with 90d delay

- Full input codes: 75.51k
- Target codes: 6.30k

Ablation config.:

- Legacy baseline: Dense + legacy config
- Exp2: Dense + optimized config (OC)
- Exp3: Dense + OC + flash attention (FA)
- Exp4: MOE + FA + OC
- One epoch

Infrastructure:

- GPU: T4 * 4 (16GB HBM each)
- Machine type: n1-highmem-64
- Data parallelism applied

Observations

- Optimized training strategy achieves over **1.4x** improvement in prediction performance
- Flash attention provides **1.5x** speedup and reduced GPU peak memory by **61%**
- MOE matched dense model performance
- **Notice**: intrinsic metrics captures only internal metrics for training quality and outcomes on predicting next day's labels; **downstream evaluation** (extrinsic) provides definite and business-driven measure of the embedding quality.

Ablation config.	Recall @10[1]	Precision @10[2]	Micro Recall@10[3]	NDCG @10[4]	Training time (H)	GPU peak memory (GB)
Random reference	0.007	0.001	0.002	0.001	NA	NA
Legacy baseline	0.579	0.120	0.234	0.169	6.15	28.29
Dense + OC	0.825	0.235	**0.478**	**0.401**	9.65[5]	20.58
Dense + OC + FA	**0.828**	**0.237**	0.462	0.398	**4.09**	**11.13**
MOE + OC + FA	0.827	0.237	0.461	0.399	4.74	13.39

1. Recall@10: fraction of members have at least 1 correct code in top-10 predictions
2. Precision@10: fraction of correct predictions at top-10 predictions
3. Micro-recall@10: fraction of true codes found in top-10 predictions
4. NDCG@10 (normalized discounted cumulative gain at top-10), indicates ranking quality, where 1 is perfect and 0 is none is relevant at top10 predictions
5. Due to the constraints of GPU memory and use of data parallelism, this experiment used 16 batch size instead of 32. The training time, given more advanced GPU, should be less than 6 hours

Lesson learnt, downstream evaluation planned

Lesson learnt

- Flash attention significantly improves the training efficiency, promising for longer patient histories with minimal computational overhead
- MOE provided marginal benefits to the model performance at current model scale; downstream task will inform final decision
- Computational resources remain a key consideration for LLM (even small) pretraining and post-training; cost estimation, governance approval and resource reservation are a must

Next step: downstream evaluation across LOBs

- Population: out of bag members (not in transformer pretraining data)
- Sampling strategy: sampled members to generate embeddings for quick around (e.g., 30%)
- Tasks across three LOBs: Commercial IP, Medicare IP and Medicaid IP
- Downstream modeling and pipeline: following production model development procedures

Migration & Adoption Strategy – Open Questions for Alignment

Current Reality

- 3 embedding pipelines running today
  - Different owners, refresh cadences
- New embedding version implies downstream model retraining
- We cannot switch embeddings without coordination

Proposed Initial Adoption Scope

- Start with a small set of downstream models to validate
  - Commercial IP Model
  - Medicaid IP Model
  - Medicare New Member Model (test Komodo value-add)

Key Questions for Alignment

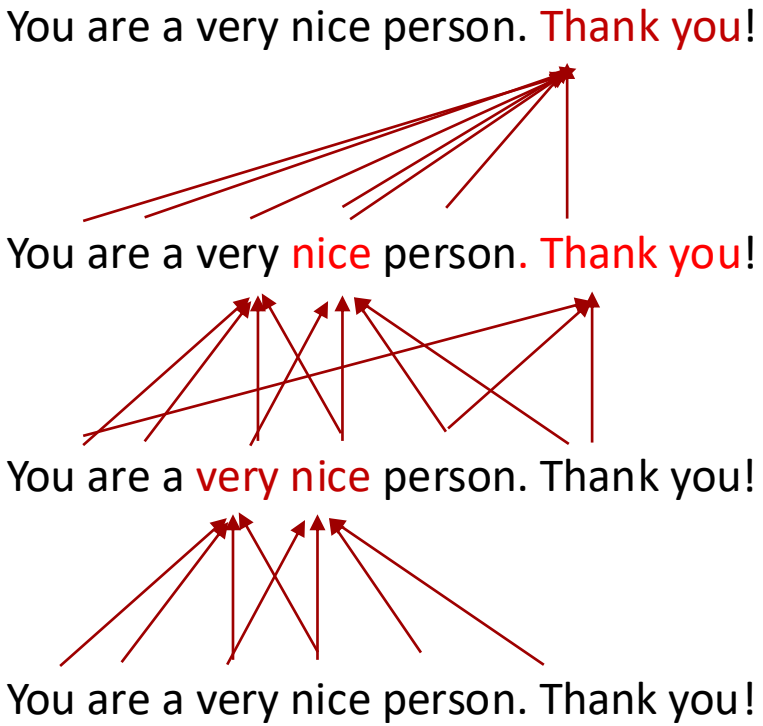
- **Evaluation/Validation Ownership**
  - A: TE Product Team
  - B: Individual model owners
- **Once adoption criteria are met, how should legacy embeddings be handled?**
  - A: Define a fixed overlap window, then deprecate
  - B: Deprecate per LOB as they migrate
  - C: Keep legacy embeddings indefinitely (not recommended)

Appendix

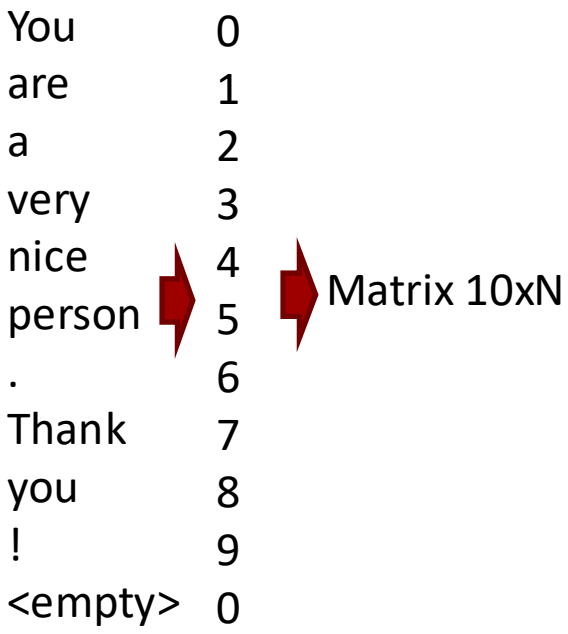
What is transformer (cont'd)

Transformer = Stacked muti-head attention + position encoding + masking.

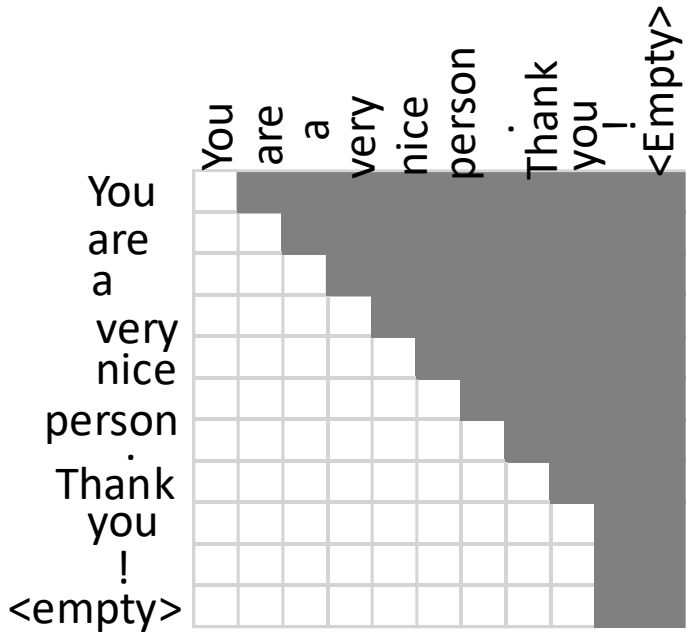
Stacked attention



Position encoding



Look ahead masking (GPT)



"Attention is all you need" Vaswani et al. (2017)

CVS Health

Slide curtesy of Min Xue

Our transformer architecture

date	icd9_dx_cd	prcdr_cd	specialty_ctg_nm	plc_srv_ctg_cd	gpi4
2021-01-29	L30.8	99213	Pediatrics	office	6699
2021-01-29	L70.9				6846
2020-10-03	Z23	90460	Pediatrics	outpatient	
		90686			
2020-03-03	J30.9	90460	Pediatrics	outpatient	
	Z00.129	90651			
	Z13.89	90686			
	Z23	96127			
	Z68.52	96160			
	Z71.3	99173			
	Z71.82	99394			
2019-04-28	R10.9	80048	Acute Short Term Hospital	emergency room	
		81001			
		82150			
		83690			
		85025			
2019-04-27	R10.9	74176	Not Mapped	emergency room	
	V86.59XA	74176			
	Y92.838	96365			
	Y93.89	99283			
		99283			
2019-04-19					6131
					6516
2019-04-02	B34.9	87880	Pediatrics	office	
		99214			
2019-01-23	Z00.129	90460	Pediatrics	office	
	Z23	90461			
	Z68.53	90715			
	Z71.3	90734			
	Z71.89	99393			
2018-12-05					120
2018-12-03	J02.9	99203	Missing	outpatient	

Individual_id =49234

Comparing to natural language, our medical data is also in sequences, but more complex:

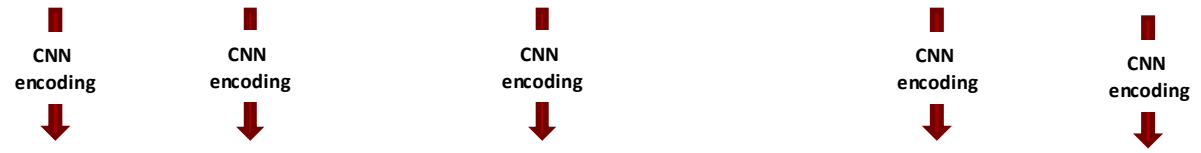
1. Considering each date is a “word”: there may be multiple data points / types at each date, such as procedure, dx, specialty etc.
2. Dates of the records are NOT continuous.
3. Different medical datasets might not align on dates, such as pharmacy data and claim data .
4. Predicting next ”word” is more challenging than NLP.

Slide curtesy of Min Xue

Our transformer architecture (cont'd)

Step1. Each date is considered as a word in a sentence. Diagnosis codes, procedure codes etc will be encoded by 1-d CNN.

date	icd9_dx_cd	prcdr_cd	specialty_ctg_nm	plc_srv_ctg_cd	gpi4
2020-03-03	J30.9	90460	Pediatrics	outpatient	
	Z00.129	90651			
	Z13.89	90686			
	Z23	96127			
	Z68.52	96160			
	Z71.3	99173			
	Z71.82	99394			



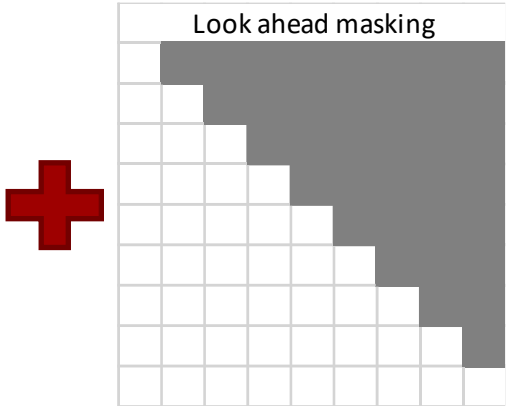
"word" Embedding = A + B + C + D + E

Slide curtesy of Min Xue

Our transformer architecture (cont'd)

Step2. Use age to encode position. Use look forward masking (like GPT), to predict next medical records, but at higher level. The paddled records are ignored in loss calculation.

date	age_in_months
2020-10-03	153.8
2020-03-03	146.8
2019-04-28	136.6
2019-04-27	136.5
2019-04-19	136.3
2019-04-02	135.7
2019-01-23	133.4
2018-12-05	131.8
2018-12-03	131.8

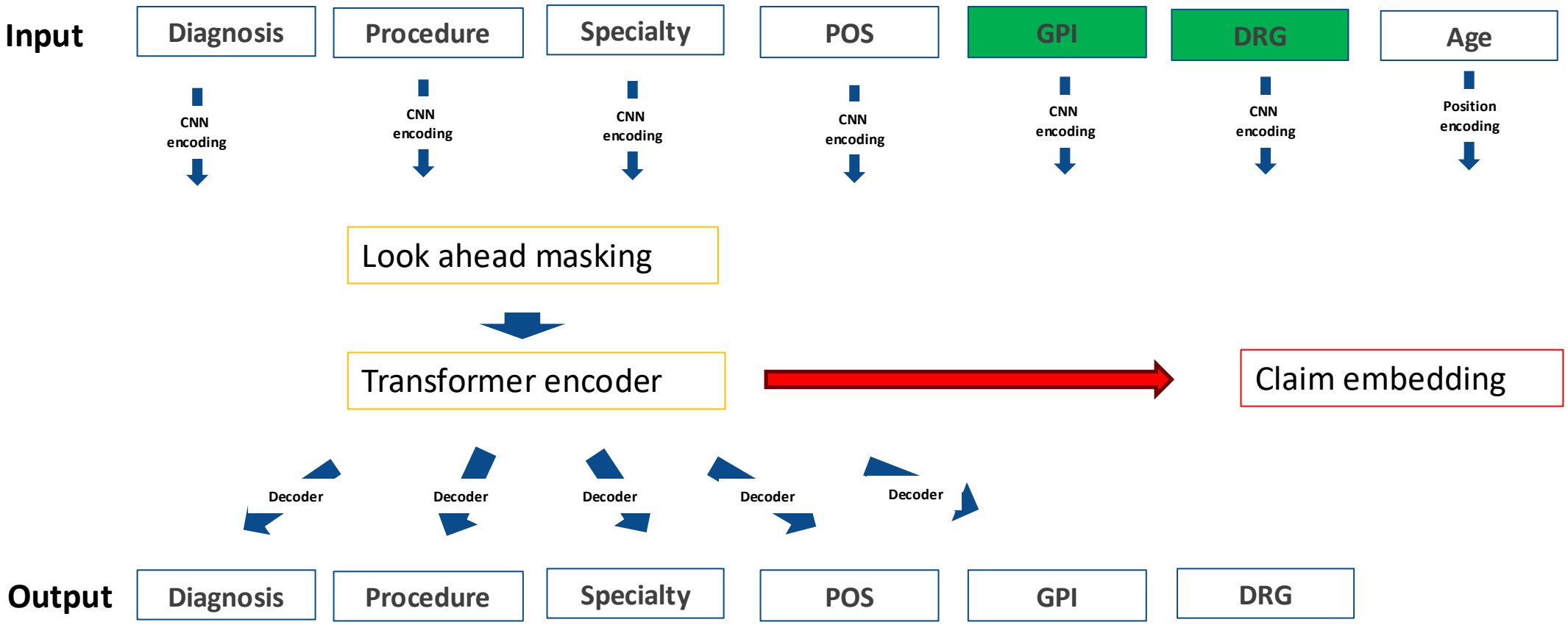


output_dx	output_procedure	output_specialty	output_pos	output_gpi
L30;L70	9921	Pediatrics	office	6699;6846
Z23	9046;9068	Pediatrics	outpatient	
J30;Z00;Z13;Z23;Z68;Z71	9046;9065;9068;9612;9616;9917;9939	Pediatrics	outpatient	
R10	8004;8100;8215;8369;8502	Acute Short Term Hospital	emergency room	
R10;V86;Y92;Y93	7417;9636;9928	Not Mapped	emergency room	
				6131;6516
B34	8788;9921	Pediatrics	office	
Z00;Z23;Z68;Z71	9046;9071;9073;9939	Pediatrics	office	
				120

Slide curtesy of Min Xue

Our transformer architecture (cont'd)

Step3. the layer right before the decoders is the embedding of the medical history, which can used as predictors in any predictive modeling. So far we have two models: claim embedding and pharmacy embedding.



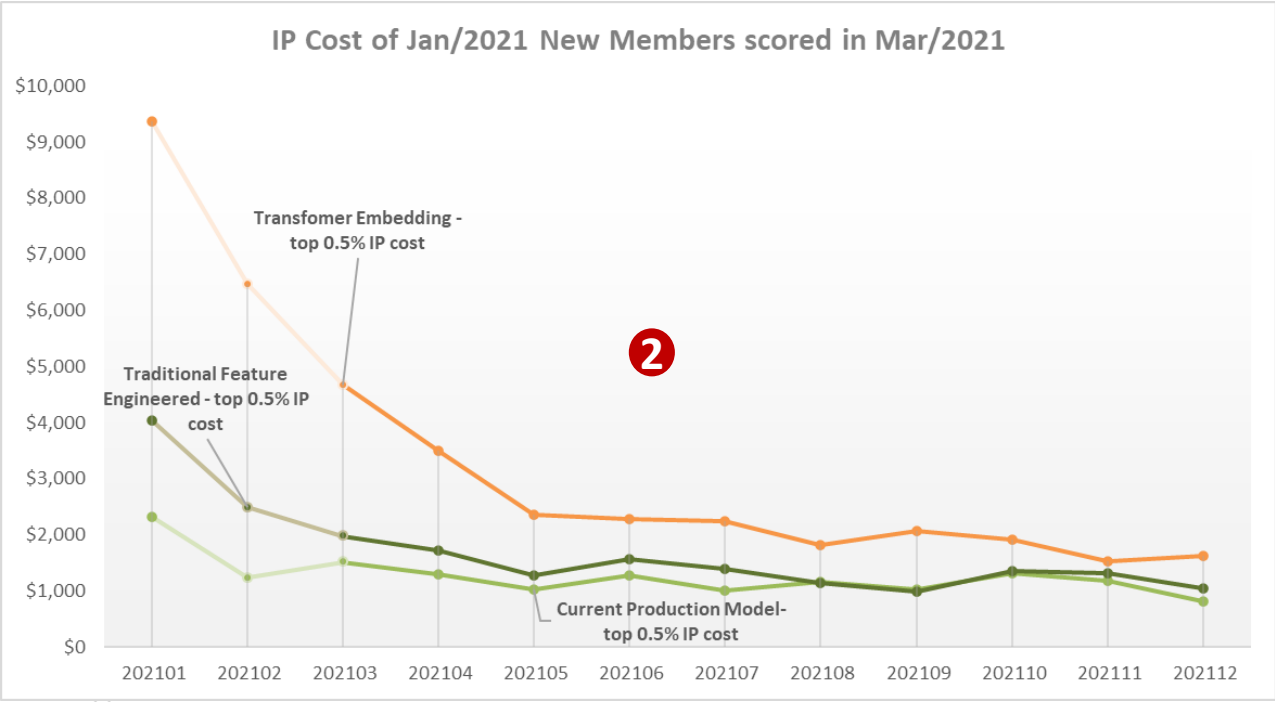
Slide curtesy of Min Xue

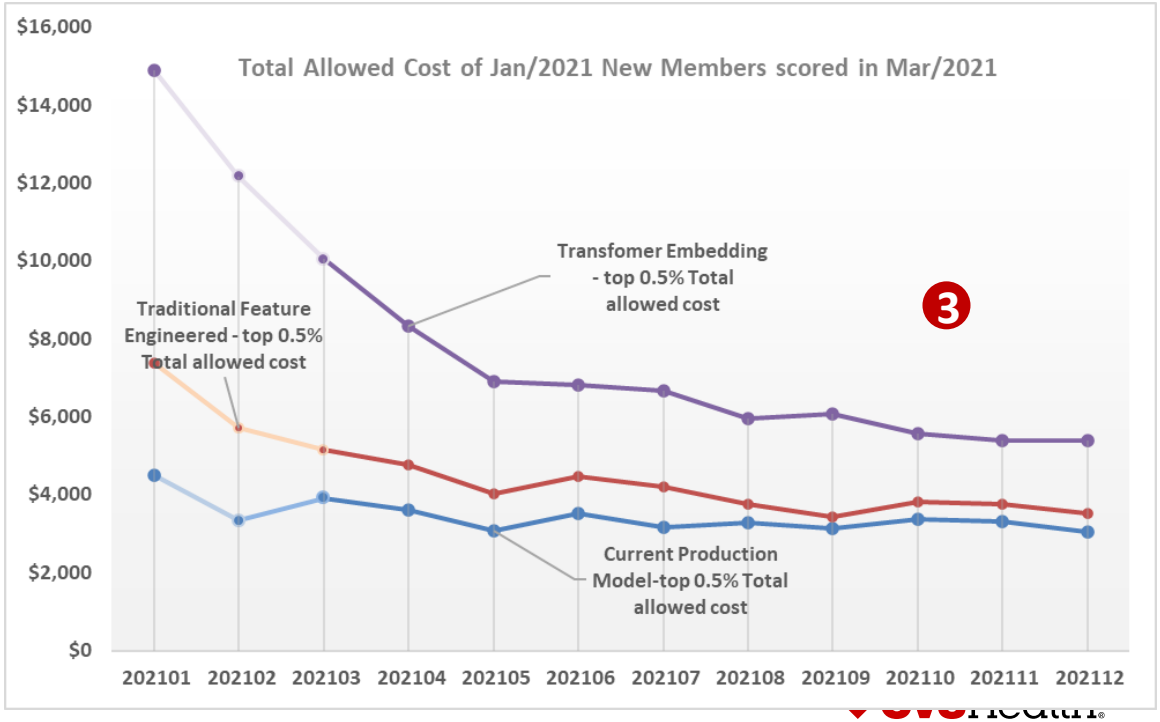
TE(v1) model outperforms traditional feature-engineered model by wide margin

Model Lift PPV (% Improvement over Production)			
Member Selection	Benchmark Legacy	Traditional Feature Engineered	Transformer Embedding
Top 0.1%	21.4	27.26 (28%)	35.88 (68%)
Top 0.5%	13.5	17.67 (31%)	22.24 (64%)
Top 1%	11.1	13.67 (24%)	16.44 (49%)

1. TE model and traditional feature engineered model perform better than the current model, TE model has the largest lift.
2. TE model identifies members with higher IP likelihood and higher IP/ER and overall medical cost (all cost categories show same pattern)
3. All 3 models indicate that member cost decreases over month after top risk identification, flattening out after month 6, suggesting urgency of care management targeting*

- **please note that due to data delay of the current process, January enrolled new members are scored in March, EDW data up to Feb is used for the March scoring, so we should look at trend starts from March*





Slide curtesy of Jane Zou, Ellen Palmer, Zenon Cheng

Medicare IP model with TE (v2)

Comparisons - based on OOT Validation		1%			10%		
	AUC	PPV	Sensitivity	Lift	PPV	Sensitivity	Lift
XGB - NRX V2 (default params)	0.78	49.0%	7.4%	8.16	25.2%	38.3%	4.20
XGB - All Features V2	0.775	48.9%	7.7%	8.15	24.4%	38.6%	4.06
XGB - All Features (default params)	0.775	48.6%	7.7%	8.10	24.3%	38.4%	4.05
Catboost - All Features	0.773	48.5%	7.7%	8.09	24.1%	38.2%	4.02
XGB - RX V2 (default params)	0.771	49.2%	7.9%	8.20	24.0%	38.5%	4.00
XGB - Embeddings + CS (minus Med/RX Claims)	0.771	47.4%	7.5%	7.90	23.9%	37.8%	3.99
XGB - NRX V2 CS Only (default params)	0.771	48.3%	7.3%	8.05	24.3%	36.8%	4.05
XGB - CS Features V2	0.768	47.3%	7.5%	7.88	23.5%	37.3%	3.92
Catboost - Embeddings + CS (minus Med/RX Claims)	0.768	47.1%	7.5%	7.86	23.8%	37.7%	3.97
XGB - CS Only (default params)	0.766	47.0%	7.4%	7.83	23.4%	37.0%	3.90
Catboost - CS Only	0.765	46.7%	7.4%	7.79	23.4%	37.0%	3.90
XGB - RX V2 CS Only (default params)	0.764	46.9%	7.5%	7.82	23.0%	34.8%	3.83
XGB - Embeddings (v2) Only	0.748	45.1%	7.1%	7.39	23.4%	36.6%	3.82
XGB - HPD Only (default params)	0.738	-	-	-	-	-	-
Production	0.732	38.1%	6.1%	6.10	20.9%	33.2%	3.34
XGB - Embeddings (v1) Only	0.717	35.8%	5.6%	6.20	20.4%	31.9%	3.36

- Embeddings did not perform as well as traditional feature engineering here, due to one key problem- RX (GPI4) was absent from the embeddings, which is crucial for the Medicare population.
- We would expect a boost to performance for embedding-only models, and perhaps a small boost to combined feature models.

Medicaid IP model with TE(v2) preliminary findings

- The TE model was trained in commercial and Medicare but still performs well in Medicaid
- Current best model in development has an AUC of ~0.87 and lift of ~20 in top 1% (data not shown)

25% holdout embeddings vs. Product Spec (Medicaid)		1%			10%		
	AUC	Precision	Recall	Lift	Precision	Recall	Lift
Logistic Regression - Embeddings only (6 month)	.84	28.9%	17%	16.98	10%	58.1%	5.81
Production (12 month)		45.1%	10.3%	10.29	19.6%	44.6%	4.46

- Breakout by tenure suggests a new member model in Medicaid is not necessary

Logistic Regression – Embeddings only (by membership length)		1%			10%		
	N (%)	Precision	Recall	Lift	Precision	Recall	Lift
0-3 Months	35,354 (6.2%)	19.8%	10.8%	10.79	6.8%	36.9%	3.69
4-8 Months	55,853 (9.8%)	26.7%	17.8%	17.77	8%	53.5%	5.35
9+ Months	477,535 (84%)	29.9%	17.8%	17.83	10.3%	61.6%	6.16

TE empowers data scientists to build better models more efficiently

Better performing model

- **Better lift/precision/AUC**
- **TE captures the intricacy of our data**, both structured and unstructured in a sequential manner, generating a condensed representation that encompasses causal effects and enhances our predictive models. This will be harder to achieve manually with thousands of sparse features. Traditional feature selection often drops features with low prevalence, such as pancreatic cancer, TE can capture rare and costly diseases.
- **Bigger business impact** Early experiments showed TE model identifying higher risk members (more IP/ER utilization, higher spend)

More Efficient

- **Time-saving.** As a data scientist, we spend up to 80% of our time cleaning and transforming data to generate actionable insights and build machine learning models.
- **Standardizes and streamlines process with claim history data.** It simplifies data scientists' work, saves ***weeks***' time and system resources, allowing us to focus on applying the latest ML skills and building models that effectively address our business goals, solve problems, and deliver value to our customers.

CVS Health

Slide curtesy of Jane Zou, Min Xue

Retraining Model – Architecture Redesign and Resource (cont')

Nvidia GPU configuration (Effective performance, TeraFLOPs/sec)[2,3] - hourly rate[1]	**Original Model** Total params: 27.7M Total FLOPs for training: 2.142 petaFLOPs[4]		**Proposed Model** Total params: 34.8M Total FLOPs for training: 2.147 petaFLOPs	
	Total estimated time	Total cost	Total estimated time	Total cost
4* T4 (41.6) - $2.99/h	1180.8 h	$3520	902.4 h	$1728
4* L4 (111.3) - $3.3/h	355.2 h	$1184	217.6 h	$704
4* A100 (561.6) - $25/h	89.6 h	$2208	41.6 h	$1072
4* H100 (1820) - $36.38/h	24.32 h	$896	12.8 h	$480

Training setup: 16M members, 2 epochs
Experimentation setup: 1-5M members, 1-2 epochs

[1] The GPU price rates are estimated by vertex AI workbench instance.
[2]FLOPs = Floating-point operations
[3]One teraFLOPs = 1 * 10^12 FLOPs
[4]One petaFLOPs = 1 * 10^15 FLOPs

Results: Probes

With probes, we can tease out which concepts the TEs are best at capturing, and where in the model the learning for a given concept set happens.

- **Key clinical concepts are captured well over the architecture**
  - The concept information accumulates over layers and the capability varied across the clinical concepts

- **The transformer encodes both shared and specialized information about concepts**
  - It starts to differentiate different clinical concepts approaching the end of layers

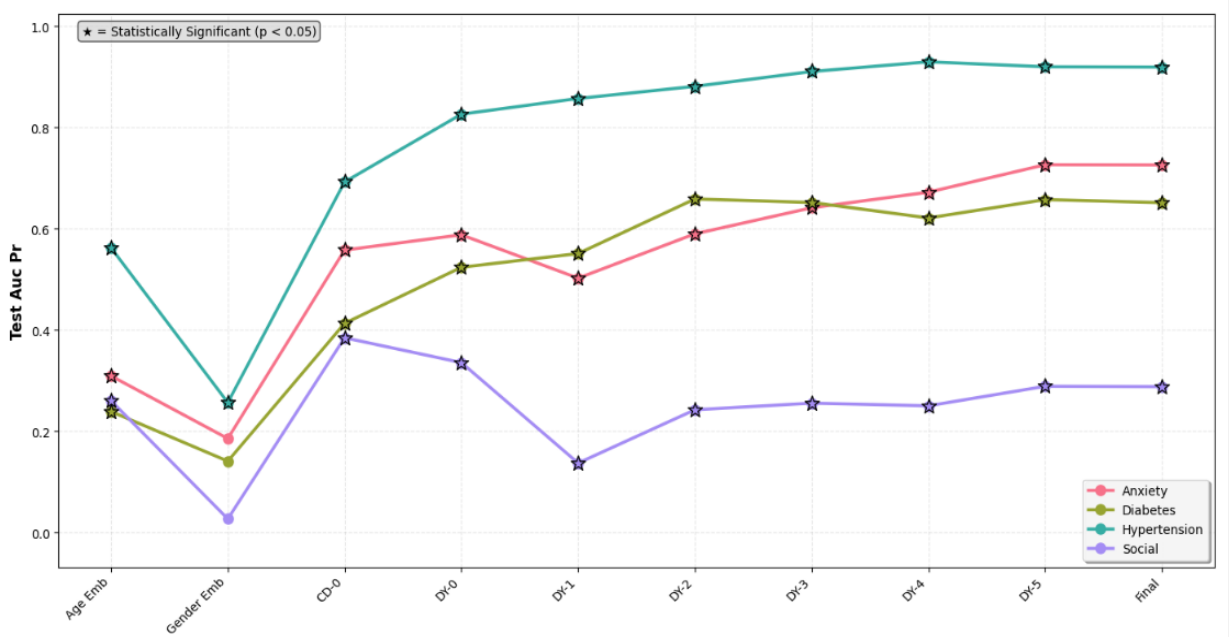


Figure 1: Performance of the transformer encoding different clinical concepts, indicated by probe classifier AUC-PR in predicting the presence of a concept within a Medicaid sampled population.

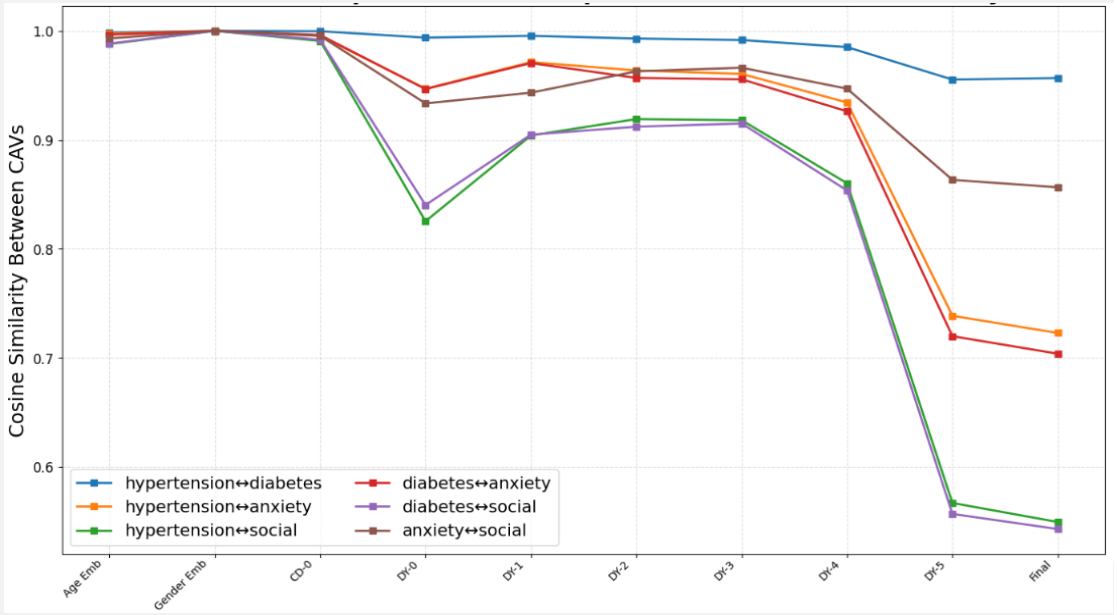


Figure 2: Cosine similarity between clinical concept vectors, indicating how much specialized information each layer encodes for a concept.